

Binairo 2026 — documentation du projet

Louis Pedelaborde, Romeo Mugnier de Almeida, Raphael Pical

État au 8 octobre 2026 : préparation du rendu. Le C et les sources/recette sont corrigés ; les VIs et la validation MATLAB/LabVIEW sur la VM restent à terminer. Ce document doit être actualisé avec les versions et résultats natifs avant remise. Références : donnée 2026.2 et addendum du 7 octobre 2026, qui fait foi pour les noms, erreurs et constantes.

Environnement et état des fichiers

Contrôle local effectué sur **macOS 27.0.1, arm64, Apple Clang 21.0.0**. Compilation C11 avec -Wall -Wextra -Wpedantic -Werror -Wvla. Le binaire bin/macos-arm64/OCR est pour cette plateforme. Le projet Visual Studio est fourni mais n’a pas été compilé sur Windows ici. La plateforme visée par la recette est la VM Windows du cours avec LabVIEW 2025 Q3 ; **les versions Windows, compilateur, MATLAB et LabVIEW réellement utilisées devront être relevées sur cette VM**. Aucune exécution native MATLAB/LabVIEW n’est attestée au 8 octobre.

Fichiers	Rôle et état
c/main.c, reconnaissance.c/.h, parametres_ocr.h	arguments, lecture, reconnaissance, écriture et contrôles ; compilés/testés
c/FontRasterized_0_1.h	police originale du professeur, inchangée
c/OCR.sln, OCR.vcxproj	projet Windows relatif, C11 ; build natif à exécuter
matlab/SolveBinairo.m	déductions puis essais récursifs 0/1, auxiliaires du squelette inclus
GrilleBinairoValide.m, FormatGrilleValide.m	règles et format des matrices
DisplayBinairo.m, CreerFigureBinairo.m	dessin et export PDF ; rendu natif à contrôler
CheminPdfBinairo.m, DispBinairo.m	nom de sortie et alias de compatibilité
BinairoSolver.vi, sous-VIs et quatre .ctl	à construire suivant la recette ; pas encore livrés comme VIs exécutables
Cell.bin, CellValue.txt, solve.m, Binairo_XXX.pdf	fichiers générés à conserver par séquence après les essais natifs
tests/VerifierMatlab.m	tests séparés et combinés MATLAB ; pas encore exécutés nativement

Inventaire des VIs à construire : **BinairoSolver**, ErreurSi, LireTexte, EcrireTexte, SupprimerResultat, TexteCheminValide, LireImage, EtendueNoire, IntervallesNoirs, ComputeCellRect, ComputeRowsCols, EcrireCellule, CommandeOCR, AnalyserResultatOCR, LireCase, GenererScript, LancerMatlab et VerifierPDF. Les quatre types sont SeuilsOCR, OptionsBinairo, LignesGrille et RectangleCellule (.ctl). Le lanceur MP_LaunchMatlabScript4 est facultatif et n’est pas une dépendance du mode direct. Chaque rôle est détaillé dans le chapitre homonyme de la recette.

Flux des données et algorithmes

PNG → LabVIEW → Cell.bin → C OCR → CellValue.txt → LabVIEW → solve.m → MATLAB → PDF. Les fichiers d'exécution sont voisins du VI principal. Les chemins se construisent à partir du VI ; aucun chemin personnel n'est inscrit dans les sources.

LabVIEW lit/convertit le PNG nativement, normalise TRUE=noir, relève les transitions sur les axes bord+10, vérifie un nombre pair et égal de cases, puis mesure chaque rectangle. Il conserve au maximum deux pixels réels du trait de chaque côté pour limiter l'influence de la bordure extérieure. Il écrit largeur puis hauteur en U32 little-endian, puis un octet 0/1 par pixel, ligne par ligne. Il appelle OCR pour chaque case et construit la matrice réellement reconnue ; -2 devient NaN. Le script MATLAB appelle directement SolveBinaire/DisplayBinaire. Les options commandent la résolution et l'ouverture du PDF.

Le C exige deux dimensions dans 50..1000, exactement largeur×hauteur octets 0/1 et trois seuils entiers 1..99. Réglage normal : 88/90/90. Le vide est reconnu en premier si le pourcentage de blanc est strictement supérieur à 88. Sinon, les deux modèles 32×32 parcourent toutes les positions possibles ; le score compte les pixels blancs et noirs égaux. Un chiffre est admissible à score>=seuil. La plus grande marge score–seuil gagne ; à égalité,0 gagne. Cette interprétation suit la procédure détaillée p. 18, malgré le résumé de score brut p. 10. Le fichier conserve le score brut, six décimales, puis une fin de ligne : d: '0' , 97.167969%. Une case vide donne d: '-2' , 0.000000%.

MATLAB vérifie format et règles, répète les déductions puis essaie 0 et 1 par récursion. Les indices initiaux sont conservés ; les fonctions de calcul n'ont pas de boucle explicite. Le PDF porte le basename du PNG, les indices noirs gras, les ajouts bleus gras, le nom en haut à gauche et la date/heure en haut à droite. Si aucune solution n'existe, == Error == apparaît au centre. Une grille insoluble produit un PDF d'erreur ; ce n'est pas une panne du lancement MATLAB.

Erreurs C et choix de warnings

Situation	Comportement
Mauvais argc, chemin invalide, seuil hors 1..99 ou syntaxe non entière	code 2, diagnostic stderr, arrêt
En-tête/dimensions invalides, pixels manquants, octet autre que 0/1	code 3, diagnostic stderr, arrêt
Lecture/ouverture/allocation impossible	code 4, diagnostic stderr, arrêt
Sortie impossible à ouvrir, écrire, vider ou fermer	code 6, diagnostic stderr, arrêt
Warning : pixels supplémentaires	choix du groupe : arrêt code 3, aucun nouveau résultat
Warning : ni vide ni chiffre admissible	choix du groupe : arrêt code 5, aucun nouveau résultat
Warning : deux chiffres admissibles	choix du groupe : sélection par marge,0 à égalité, code 0 ; pas de message de warning non fatal

Tous les cas d'arrêt ont un code non nul. LabVIEW supprime l'ancien résultat avant l'appel et vérifie le code/cluster d'erreur ; il ne lit pas un ancien CellValue après échec. Le C peut laisser un fichier précédent si la reconnaissance échoue avant l'ouverture de la sortie. Un échec pendant l'écriture peut laisser un fichier incomplet : toujours vérifier le code. À succès, stdout/stderr sont vides. Les pixels sont alloués par malloc puis libérés sur tous les chemins ; aucun VLA.

Erreurs d'orchestration et validation

Les sous-VIs propagent le premier cluster d'erreur. La recette vérifie notamment PNG absent/invalid (y compris BadBinaïro.png), géométrie invalide, OCR absent/en erreur, sortie ou script non inscriptible, MATLAB absent/script en erreur, PDF absent/vidé et résultats périmés. Les codes 6101..6105 et 7001..7012 sont des choix locaux, pas une numérotation imposée. Les erreurs natives peuvent conserver leurs codes. La fin de System Exec doit être attendue ; le PDF est contrôlé après un retour réussi.

Obtenu localement : 126 appels du vrai C, sans écart aux résultats attendus ; six passages représentatifs avec ASan/UBSan sans diagnostic ; 16 contrôles directs du parseur, 8 de sélection et 2048 bits de police. Le découpage révisé, avec quatre conversions RGB d'essai, donne 464 reconnaissances correctes : 29 indices et 87 vides par conversion. Le 5×6 est refusé. La conversion RGB de ce contrôle indépendant n'est pas l'exécution du convertisseur NI.

Encore requis sur la VM : compiler OCR.exe, construire les VIs, exécuter VerifierMatlab et les checkpoints/essais V01-V25, inspecter les PDF, sauvegarder les défauts 88/90/90 et produire les fichiers natifs par séquence. Conserver tous les fichiers projet, résultats et transcripts IA PDF dans l'archive nommée d'après les membres ; ne pas inclure les PNG d'entrée. Retélécharger/extraire l'archive dans un autre dossier ou sur une autre machine et vérifier l'exécution par la flèche Run.

Références et aide utilisée

La police et les en-têtes du solveur proviennent du professeur. Les techniques se rattachent à ME-213 et CS-119a dans Conformite_programme.md. Les références NI/MathWorks concernent les bornes/options natives et sont listées dans la recette. L'audit et ces corrections ont reçu une aide IA ; **joindre les conversations complètes en PDF** au rendu. Toute autre source de code ou coopération significative avec un autre groupe doit également être déclarée par les auteurs. Aucun transcript ni résultat natif n'est fabriqué pour combler une pièce manquante.